

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №2.

Тема: «Бинарные деревья»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Необходимо реализовать функции для редактирования дерева:

- Вставка узла.
- Удаление узла.
- Поиск элемента по ключу.

2. Реализовать алгоритмы обхода дерева:

2.1 Прямой

2.2 Симметричный

2.3 Обратный

3. Реализовать алгоритм балансировки дерева.

Анализ задачи:

Проект разделен на три логических уровня (паттерн Model-View-Controller):

Model (Graph класс): Отвечает только за логику. Хранит данные (списки смежности) и выполняет математические расчеты алгоритмов. Не знает ничего о кнопках и окнах.

View (GraphWidget): Отвечает за отрисовку. Использует QPainter для рисования линий и эллипсов. Принимает команды о цветах (какую вершину подсветить красным, а какую зеленым).

Controller (MainWindow): Связывает UI и Модель. Обрабатывает нажатия кнопок, запускает таймер анимации (QTimer) и обновляет статус.

Код:

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(BSTVisualizer)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
```

```

set(CMAKE_AUTOUIC ON)

# ВАЖНО: укажи свой путь к MinGW версии Qt
set(CMAKE_PREFIX_PATH "C:/Qt/6.11.1/mingw_64")

find_package(Qt6 COMPONENTS Widgets REQUIRED)

add_executable(BSTVisualizer
    main.cpp
    mainwindow.cpp
    treewidget.cpp
    binarytree.cpp
)

target_link_libraries(BSTVisualizer Qt6::Widgets)

```

treenode.h

```

#ifndef TREENODE_H
#define TREENODE_H

#include <QPointF>

struct TreeNode {
    int key;
    TreeNode* left;
    TreeNode* right;
    int height; // Для AVL

    // Для визуализации

```

```

    QPointF pos;

    int depth;

    TreeNode(int k) : key(k), left(nullptr),
right(nullptr), height(1), pos(0,0), depth(0) {}

};

#endif // TREENODE_H

```

binarytree.cpp

```

#include "binarytree.h"
#include <algorithm>
#include <queue>

BinaryTree::BinaryTree() : root(nullptr),
inOrderIndex(0) {}

BinaryTree::~BinaryTree() { clear(); }

void BinaryTree::clear() {
    if (!root) return;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* n = q.front(); q.pop();
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
        delete n;
    }
    root = nullptr;
}

```

```

}

// === INSERT / REMOVE / SEARCH ===

void BinaryTree::insert(int key) {
    root = insertRec(root, key);
    updateLayout();
}

TreeNode* BinaryTree::insertRec(TreeNode* node, int
key) {
    if (!node) return new TreeNode(key);
    if (key < node->key) node->left =
insertRec(node->left, key);
    else if (key > node->key) node->right =
insertRec(node->right, key);
    else return node; // Дубликаты игнорируем

    node->height = 1 + std::max(getHeight(node->left),
getHeight(node->right));

    int balance = getBalance(node);

    if (balance > 1 && key < node->left->key) return
rotateRight(node);

    if (balance < -1 && key > node->right->key) return
rotateLeft(node);

    if (balance > 1 && key > node->left->key) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }

    if (balance < -1 && key < node->right->key) {
        node->right = rotateRight(node->right);

```

```

        return rotateLeft(node);
    }

    return node;
}

void BinaryTree::remove(int key) {
    root = removeRec(root, key);
    updateLayout();
}

TreeNode* BinaryTree::removeRec(TreeNode* node, int
key) {
    if (!node) return node;

    if (key < node->key) node->left =
removeRec(node->left, key);

    else if (key > node->key) node->right =
removeRec(node->right, key);

    else {
        if (!node->left || !node->right) {
            TreeNode* temp = node->left ? node->left :
node->right;

            if (!temp) { temp = node; node = nullptr; }
            else *node = *temp;

            delete temp;
        } else {
            TreeNode* temp = minValueNode(node->right);
            node->key = temp->key;
            node->right = removeRec(node->right,
temp->key);
        }
    }
}

```

```

    }

    if (!node) return node;

    node->height = 1 + std::max(getHeight(node->left),
getHeight(node->right));

    int balance = getBalance(node);

    if (balance > 1 && getBalance(node->left) >= 0)
return rotateRight(node);

    if (balance > 1 && getBalance(node->left) < 0) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }

    if (balance < -1 && getBalance(node->right) <= 0)
return rotateLeft(node);

    if (balance < -1 && getBalance(node->right) > 0) {
        node->right = rotateRight(node->right);
        return rotateLeft(node);
    }

    return node;
}

```

```

TreeNode* BinaryTree::minValueNode(TreeNode* node) {
    TreeNode* current = node;
    while (current->left) current = current->left;
    return current;
}

```

```

TreeNode* BinaryTree::search(int key) {
    TreeNode* curr = root;

```

```

        while (curr) {
            if (key == curr->key) return curr;
            curr = (key < curr->key) ? curr->left :
curr->right;
        }
        return nullptr;
    }

// === TRAVERSALS ===

std::vector<int> BinaryTree::preOrder() {
    std::vector<int> res;

    traverse(root, [&res] (TreeNode* n) {
res.push_back(n->key); }, 0);

    return res;
}

std::vector<int> BinaryTree::inOrder() {
    std::vector<int> res;

    traverse(root, [&res] (TreeNode* n) {
res.push_back(n->key); }, 1);

    return res;
}

std::vector<int> BinaryTree::postOrder() {
    std::vector<int> res;

    traverse(root, [&res] (TreeNode* n) {
res.push_back(n->key); }, 2);

    return res;
}

void BinaryTree::traverse(TreeNode* node,
std::function<void(TreeNode*)> visit, int type) {

```

```

        if (!node) return;
        if (type == 0) visit(node);
        traverse(node->left, visit, type);
        if (type == 1) visit(node);
        traverse(node->right, visit, type);
        if (type == 2) visit(node);
    }

    // === BALANCING ===

    void BinaryTree::balance() {
        // Собираем все ключи, удаляем дерево, вставляем
        // заново (автоматическая AVL-балансировка при insert)
        std::vector<TreeNode*> nodes = getAllNodes();
        std::vector<int> keys;
        for(auto n : nodes) keys.push_back(n->key);
        clear();
        for(int k : keys) insert(k);
    }

    std::vector<TreeNode*> BinaryTree::getAllNodes() {
        std::vector<TreeNode*> list;
        collectNodes(root, list);
        return list;
    }

    void BinaryTree::collectNodes(TreeNode* node,
        std::vector<TreeNode*>& list) {
        if (!node) return;
        list.push_back(node);
        collectNodes(node->left, list);
        collectNodes(node->right, list);
    }

```

```

}

// === LAYOUT CALCULATION ===

void BinaryTree::updateLayout() {
    if (!root) return;
    inOrderIndex = 0;
    assignInOrderIndex(root);
    assignCoords(root, 50.0, 60.0);
}

void BinaryTree::assignInOrderIndex(TreeNode* node) {
    if (!node) return;
    assignInOrderIndex(node->left);
    node->pos.setX(inOrderIndex++);
    assignInOrderIndex(node->right);
}

void BinaryTree::assignCoords(TreeNode* node, double
xGap, double yGap) {
    if (!node) return;
    node->pos.setX(node->pos.x() * xGap);
    node->pos.setY(node->depth * yGap);
    if (node->left) { node->left->depth = node->depth +
1; assignCoords(node->left, xGap, yGap); }
    if (node->right) { node->right->depth = node->depth
+ 1; assignCoords(node->right, xGap, yGap); }
}

// === AVL HELPERS ===

```

```

int BinaryTree::getHeight(TreeNode* n) { return n ?
n->height : 0; }

int BinaryTree::getBalance(TreeNode* n) { return n ?
getHeight(n->left) - getHeight(n->right) : 0; }

TreeNode* BinaryTree::rotateRight(TreeNode* y) {
    TreeNode* x = y->left;
    TreeNode* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = std::max(getHeight(y->left),
getHeight(y->right)) + 1;
    x->height = std::max(getHeight(x->left),
getHeight(x->right)) + 1;
    return x;
}

TreeNode* BinaryTree::rotateLeft(TreeNode* x) {
    TreeNode* y = x->right;
    TreeNode* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = std::max(getHeight(x->left),
getHeight(x->right)) + 1;
    y->height = std::max(getHeight(y->left),
getHeight(y->right)) + 1;
    return y;
}

```

treewidget.cpp

```
#include "treewidget.h"
```

```
TreeWidget::TreeWidget(QWidget* parent) :
QWidget(parent), tree(nullptr), highlightNode(nullptr)
{
    setMinimumSize(600, 400);
    setMouseTracking(true);
}

void TreeWidget::setTree(BinaryTree* t) {
    tree = t;
    update();
}

void TreeWidget::setHighlightNode(TreeNode* node) {
    highlightNode = node;
    update();
}

void TreeWidget::setVisitedNodes(const
std::vector<TreeNode*>& nodes) {
    visitedNodes = nodes;
    update();
}

void TreeWidget::setSearchPath(const
std::vector<TreeNode*>& path) {
    searchPath = path;
    update();
}
```

```

void TreeWidget::paintEvent(QPaintEvent* event) {
    QWidget::paintEvent(event);

    if (!tree || !tree->getRoot()) return;

    QPainter p(this);
    p.setRenderHint(QPainter::Antialiasing);

    // === ИСПРАВЛЕНИЕ: Добавляем отступ, чтобы дерево
не обрезалось ===
    p.translate(40, 40);

    auto nodes = tree->getAllNodes();
    for (auto n : nodes) {
        if (n->left) drawEdge(p, n, n->left);
        if (n->right) drawEdge(p, n, n->right);
    }
    for (auto n : nodes) drawNode(p, n);
}

void TreeWidget::drawEdge(QPainter& p, TreeNode*
parent, TreeNode* child) {
    bool isSearch = std::find(searchPath.begin(),
searchPath.end(), parent) != searchPath.end() &&
        std::find(searchPath.begin(),
searchPath.end(), child) != searchPath.end();

    p.setPen(QPen(isSearch ? QColor(255, 87, 34) :
QColor(150, 150, 150), isSearch ? 3 : 2));
    p.drawLine(parent->pos, child->pos);
}

```

```

void TreeWidget::drawNode(QPainter& p, TreeNode* node)
{
    int radius = 20;

    QColor color = QColor(33, 150, 243); // Синий по
умолчанию

    if (node == highlightNode) color = QColor(255, 87,
34); // Оранжевый

    else if (std::find(visitedNodes.begin(),
visitedNodes.end(), node) != visitedNodes.end()) color
= QColor(76, 175, 80); // Зеленый

    else if (std::find(searchPath.begin(),
searchPath.end(), node) != searchPath.end()) color =
QColor(255, 193, 7); // Желтый

    // Тень для выделения
    if (node == highlightNode) {
        p.setBrush(QColor(255, 235, 59));
        p.drawEllipse(node->pos, radius+4, radius+4);
    }

    p.setBrush(color);
    p.setPen(QPen(Qt::black, 2));
    p.drawEllipse(node->pos, radius, radius);

    p.setPen(Qt::white);
    p.setFont(QFont("Arial", 12, QFont::Bold));
    p.drawText(QRectF(node->pos.x()-radius,
node->pos.y()-radius, radius*2, radius*2),
                Qt::AlignCenter,
QString::number(node->key));

```

```

}

void TreeWidget::mouseMoveEvent(QMouseEvent* event) {
    // Можно добавить тултипы или перетаскивание, но
    // для дерева авто-лейаут удобнее
    QWidget::mouseMoveEvent(event);
}

```

mainwindow.cpp

```

#include "mainwindow.h"
#include <QMessageBox>

MainWindow::MainWindow(QWidget* parent) :
    QMainWindow(parent), timer(new QTimer(this)) {
    tree = new BinaryTree();
    setupUI();

    connect(timer, &QTimer::timeout, this,
        &MainWindow::animationStep);

    // Стартовый набор для демонстрации
    tree->insert(50); tree->insert(30);
    tree->insert(70);

    tree->insert(20); tree->insert(40);
    tree->insert(60); tree->insert(80);

    tree->insert(10); // Вызовет балансировку AVL
    treeWidget->setTree(tree);
}

MainWindow::~MainWindow() { delete tree; }

```

```
void MainWindow::setupUI() {  
    setWindowTitle("BST & AVL Visualizer");  
    resize(1000, 700);  
  
    QWidget* central = new QWidget(this);  
    setCentralWidget(central);  
    QVBoxLayout* main = new QVBoxLayout(central);  
  
    // === Управление ===  
    QHBoxLayout* ctrl = new QHBoxLayout();  
    keyEdit = new QLineEdit();  
    keyEdit->setPlaceholderText("Ключ");  
    keyEdit->setMaximumWidth(80);  
    ctrl->addWidget(keyEdit);  
  
    // 1. Создаем кнопки как переменные  
    auto* btnInsert = new QPushButton("Вставить");  
    ctrl->addWidget(btnInsert);  
  
    auto* btnRemove = new QPushButton("Удалить");  
    ctrl->addWidget(btnRemove);  
  
    auto* btnSearch = new QPushButton("Поиск");  
    ctrl->addWidget(btnSearch);  
    main->addLayout(ctrl);  
  
    // 2. Подключаем сигналы к переменным
```

```
    connect(btnInsert, &QPushButton::clicked, this,
&MainWindow::insertNode);

    connect(btnRemove, &QPushButton::clicked, this,
&MainWindow::removeNode);

    connect(btnSearch, &QPushButton::clicked, this,
&MainWindow::searchNode);


// === Алгоритмы ===

QHBoxLayout* algo = new QHBoxLayout();

// Здесь твой код с лямбдой был правильным,
оставляем как есть

auto addBtn = [&](const QString& txt, auto slot){
    auto* b = new QPushButton(txt);
    b->setStyleSheet("padding:5px;");
    algo->addWidget(b);
    connect(b, &QPushButton::clicked, this, slot);
};

addBtn("Прямой (Pre)", &MainWindow::startPreOrder);
addBtn("Симметричный (In)",
&MainWindow::startInOrder);

addBtn("Обратный (Post)",
&MainWindow::startPostOrder);

addBtn("Балансировка (AVL)",
&MainWindow::startBalance);

main->addLayout(algo);

statusLabel = new QLabel("Готов");

statusLabel->setStyleSheet("background:#E3F2FD;
padding:8px; font-weight:bold;");
```

```
main->addWidget(statusLabel);

treeWidget = new TreeWidget();
treeWidget->setTree(tree);
main->addWidget(treeWidget, 1);

resultLabel = new QLabel("Результат:");
resultLabel->setWordWrap(true);
resultLabel->setStyleSheet("background:#f5f5f5;
padding:10px; font-family:monospace;");
main->addWidget(resultLabel);
}

void MainWindow::insertNode() {
    bool ok; int k = keyEdit->text().toInt(&ok);
    if(!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите число");
        return; }
    tree->insert(k);
    treeWidget->setTree(tree);
    statusLabel->setText("Вставлен узел: " +
        QString::number(k));
    keyEdit->clear();
}

void MainWindow::removeNode() {
    bool ok; int k = keyEdit->text().toInt(&ok);
    if(!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите число");
        return; }
    tree->remove(k);
```

```
        treeWidget->setTree(tree);

        statusLabel->setText("Удален узел: " +
QString::number(k));

        keyEdit->clear();
    }

void MainWindow::searchNode() {
    bool ok; int k = keyEdit->text().toInt(&ok);

    if(!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите число");
        return;
    }

    // Находим путь поиска для анимации
    std::vector<TreeNode*> path;

    TreeNode* curr = tree->getRoot();
    while(curr) {
        path.push_back(curr);

        if(k == curr->key) break;

        curr = (k < curr->key) ? curr->left :
curr->right;
    }

    mode = SEARCH;

    animSequence = path;

    currentStep = 0;

    statusLabel->setText("Поиск узла " +
QString::number(k));

    timer->stop();

    connect(timer, &QTimer::timeout, this,
&MainWindow::animationStep);
}
```

```
        timer->start(800);
    }

void MainWindow::startPreOrder() {
    mode = PRE;
    auto keys = tree->preOrder();
    animSequence.clear();
    for(int k : keys) {
        TreeNode* n = tree->search(k);
        if(n) animSequence.push_back(n);
    }
    currentStep = 0;
    statusLabel->setText("Прямой обход (Pre-Order)");
    timer->stop(); connect(timer, &QTimer::timeout,
this, &MainWindow::animationStep); timer->start(800);
}

void MainWindow::startInOrder() {
    mode = IN;
    auto keys = tree->inOrder();
    animSequence.clear();
    for(int k : keys) {
        TreeNode* n = tree->search(k);
        if(n) animSequence.push_back(n);
    }
    currentStep = 0;
    statusLabel->setText("Симметричный обход (In-Order)");
    timer->stop(); connect(timer, &QTimer::timeout,
this, &MainWindow::animationStep); timer->start(800);
}
```

```

}

void MainWindow::startPostOrder() {
    mode = POST;
    auto keys = tree->postOrder();
    animSequence.clear();
    for(int k : keys) {
        TreeNode* n = tree->search(k);
        if(n) animSequence.push_back(n);
    }
    currentStep = 0;
    statusLabel->setText("Обратный обход  
(Post-Order)");
    timer->stop(); connect(timer, &QTimer::timeout,
this, &MainWindow::animationStep); timer->start(800);
}

void MainWindow::startBalance() {
    mode = BALANCE;
    statusLabel->setText("AVL Балансировка...");
    tree->balance();
    treeWidget->setTree(tree);
    treeWidget->setVisitedNodes(tree->getAllNodes());
    resultLabel->setText("Дерево перестроено и  
сбалансировано по AVL.");
    timer->stop();
}

void MainWindow::animationStep() {
    if(currentStep >= animSequence.size()) {

```

```

        timer->stop();

        statusLabel->setText("Анимация завершена");

        treeWidget->setHighlightNode(nullptr);

        QString res = "Порядок: ";

        for(auto n : animSequence) res +=
QString::number(n->key) + " ";

        resultLabel->setText(res);

        return;
    }

    TreeNode* node = animSequence[currentStep];

    treeWidget->setHighlightNode(node);

    std::vector<TreeNode*>
visited(animSequence.begin(), animSequence.begin() +
currentStep + 1);

    treeWidget->setVisitedNodes(visited);

    if(mode == SEARCH)
treeWidget->setSearchPath(visited);

    currentStep++;
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

```

```
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QPushButton>
#include <QLineEdit>
#include <QLabel>
#include <QTimer>
#include "treewidget.h"
#include "binarytree.h"

class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    MainWindow(QWidget* parent = nullptr);
    ~MainWindow();

private slots:
    void insertNode();
    void removeNode();
    void searchNode();

    void startPreOrder();
    void startInOrder();
    void startPostOrder();

    void startBalance();

    void animationStep();

private:
```

```

void setupUI();

BinaryTree* tree;
TreeWidget* treeWidget;

QLineEdit* keyEdit;
QLabel* statusLabel;
QLabel* resultLabel;

QTimer* timer;

// Состояние анимации
enum AnimMode { NONE, PRE, IN, POST, SEARCH, BALANCE };
AnimMode mode;
std::vector<TreeNode*> animSequence;
int currentStep;
};

#endif // MAINWINDOW_H

```

treewidget.h

```

#ifndef TREETWIDGET_H
#define TREETWIDGET_H

#include <QWidget>
#include <QPainter>
#include "binarytree.h"

```

```

class TreeWidget : public QWidget {
    Q_OBJECT
public:
    explicit TreeWidget(QWidget* parent = nullptr);
    void setTree(BinaryTree* tree);
    void setHighlightNode(TreeNode* node);
    void setVisitedNodes(const std::vector<TreeNode*>&
nodes);
    void setSearchPath(const std::vector<TreeNode*>&
path);

protected:
    void paintEvent(QPaintEvent* event) override;
    void mouseMoveEvent(QMouseEvent* event) override;

private:
    BinaryTree* tree;
    TreeNode* highlightNode;
    std::vector<TreeNode*> visitedNodes;
    std::vector<TreeNode*> searchPath;

    void drawNode(QPainter& p, TreeNode* node);
    void drawEdge(QPainter& p, TreeNode* parent,
TreeNode* child);
};

#endif // TREETWIDGET_H

```

binarytree.h

```

#ifndef BINARYTREE_H

```

```

#define BINARYTREE_H

#include "treenode.h"
#include <vector>
#include <functional>

class BinaryTree {
public:
    BinaryTree();
    ~BinaryTree();

    void clear();
    void insert(int key);
    void remove(int key);
    TreeNode* search(int key);

    std::vector<int> preOrder();
    std::vector<int> inOrder();
    std::vector<int> postOrder();

    void balance(); // AVL балансировка
    void updateLayout(); // Пересчет координат для
отрисовки

    TreeNode* getRoot() const { return root; }
    std::vector<TreeNode*> getAllNodes();

private:
    TreeNode* root;

```

```

// Вспомогательные для AVL
int getHeight(TreeNode* n);
int getBalance(TreeNode* n);
TreeNode* rotateRight(TreeNode* y);
TreeNode* rotateLeft(TreeNode* x);
TreeNode* insertRec(TreeNode* node, int key);
TreeNode* removeRec(TreeNode* node, int key);
TreeNode* minValueNode(TreeNode* node);

// Вспомогательные для обходов
void traverse(TreeNode* node,
std::function<void(TreeNode*)> visit, int type); //
0=pre, 1=in, 2=post
void collectNodes(TreeNode* node,
std::vector<TreeNode*>& list);

// Вспомогательные для layout
int inOrderIndex;
void assignInOrderIndex(TreeNode* node);
void assignCoords(TreeNode* node, double xGap,
double yGap);
};

#endif // BINARYTREE_H

```

main.cpp

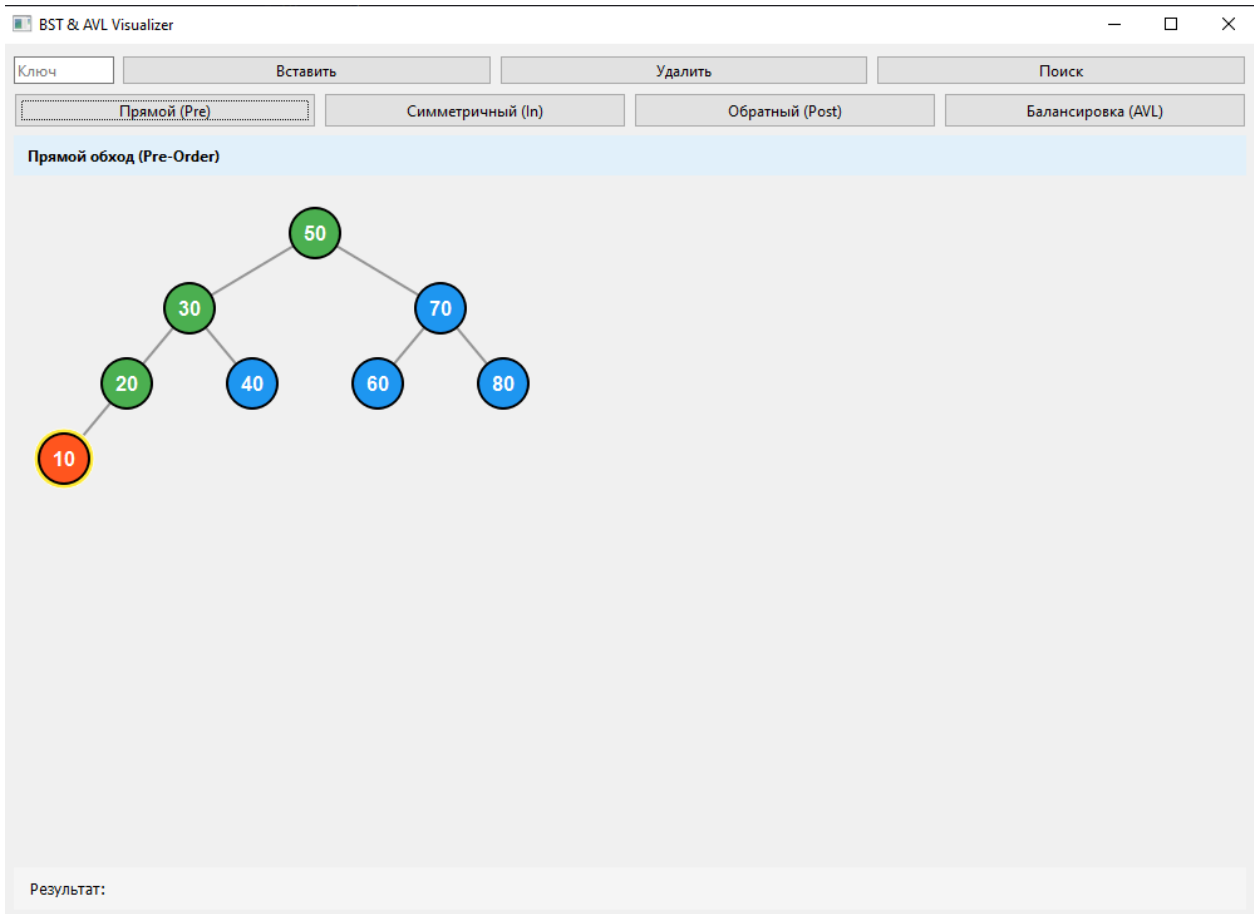
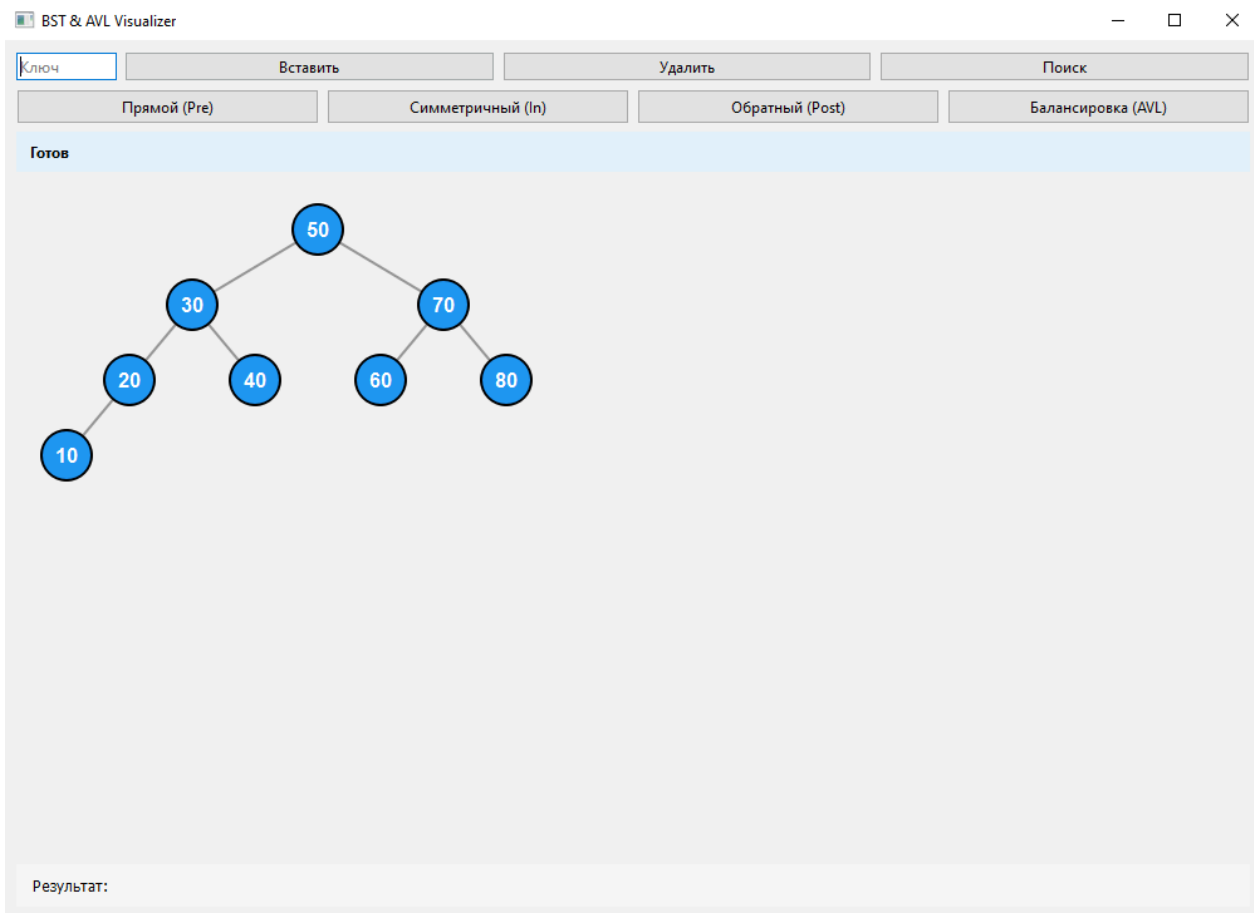
```

#include <QApplication>
#include "mainwindow.h"

```

```
int main(int argc, char *argv[]) {  
    QApplication a(argc, argv);  
    MainWindow w;  
    w.show();  
    return a.exec();  
}
```

Результаты работы:



UML

